CENG 305 – Operating Systems

Process Scheduling Simulator

Name: İpek Demirel

Student ID: 220446049

Department: Software Engineering

1. Introduction:

The objective of this assignment is to gain a deeper understanding of CPU scheduling algorithms by implementing them in a simulated environment. In this project, a process scheduling simulator was developed to model how different scheduling policies affect process execution and system performance.

The simulator supports four scheduling algorithms: First-Come First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin. The program reads process information from an input file and computes important performance metrics such as turnaround time, waiting time, and CPU utilization.

In addition to a command-line interface, a graphical user interface (GUI) was developed using Python's Tkinter library. The GUI allows users to easily load input files, configure the Round Robin time quantum, and visualize results for each scheduling algorithm.

2. Design:

The simulator is implemented in Python using a modular design. Each process is represented by a Process class that stores the process ID, arrival time, burst time, and priority.

The scheduling algorithms are implemented as separate functions. A simulation clock is maintained to track the current CPU time. A ready queue is used to select the next process to execute based on the scheduling policy. Idle CPU periods are explicitly handled when no process is available, and these periods are included in the Gantt chart.

For each algorithm, a Gantt chart is generated to visually represent the execution order of processes on the CPU. Finish time, turnaround time, and waiting time are calculated for every process.

The project supports both a command-line interface and a graphical user interface (GUI). The GUI is implemented using Tkinter and provides a user-friendly way to run all scheduling algorithms. It includes separate tabs for FCFS, SJF, Priority, and Round Robin scheduling, as well as a summary tab that displays average metrics and CPU utilization for easy comparison.

3. Results and Analysis:

The simulator was tested using the sample input file provided in the assignment instructions. All four scheduling algorithms were executed, and Round Robin was tested with a time quantum of 3.

The results indicate that Shortest Job First (SJF) achieved the lowest average turnaround time and waiting time for this specific input. FCFS showed higher waiting times for processes arriving later, while Priority Scheduling produced similar results to SJF due to the distribution of priorities.

Round Robin scheduling resulted in higher average turnaround and waiting times because of frequent preemption, but it ensured fairness by allowing all processes to receive CPU time. The summary table generated by the simulator provides a clear comparison of all algorithms and their performance metrics.

The summary table generated by the simulator clearly presents the average

turnaround time, average waiting time, and CPU utilization for all scheduling algorithms, enabling an effective comparison of their performance.

4. Discussion:

For the given input, the Shortest Job First (SJF) algorithm performed best in terms of average turnaround time and waiting time. This is because SJF prioritizes shorter processes, allowing them to complete earlier and reducing overall waiting time in the system.

Round Robin scheduling involves a trade-off between performance and fairness. While it prevents any single process from monopolizing the CPU, frequent context switches increase waiting and turnaround times. A smaller time quantum improves responsiveness but increases overhead, whereas a larger time quantum makes the algorithm behave similarly to FCFS.

Starvation is a situation where a process waits indefinitely for CPU time. Priority Scheduling can lead to starvation when low-priority processes are repeatedly postponed by higher-priority processes. This issue was demonstrated using the starvation.txt input file, where a low-priority process experienced excessive waiting due to the continuous execution of higher-priority processes.

In real-world systems, processes often perform I/O operations. The presence of I/O bursts would generally favor Round Robin scheduling, as I/O-bound processes frequently release the CPU and benefit from time slicing. In contrast, SJF may perform poorly if CPU burst times are difficult to predict due to I/O behavior.